

PROCESSES vs THREADS

ENGINEERING MAP FOR SENIOR SOFTWARE ENGINEER / TECHNICAL LEAD INTERVIEW PREP

1 WHY

Choose the right unit of execution and isolation.

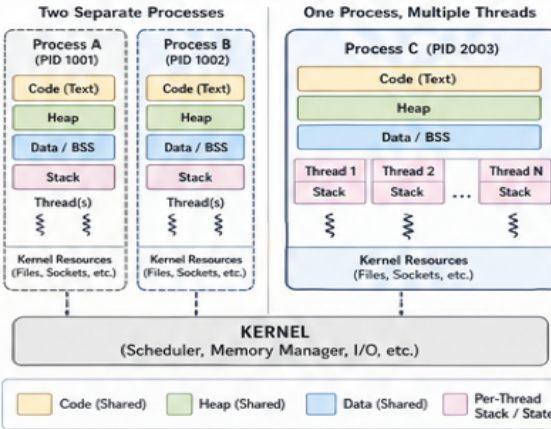
	ISOLATION Separate failures, crashes, and security boundaries.
	PARALLELISM Use multiple CPU cores for true parallel execution.
	RESPONSIVENESS Keep systems responsive under load and blocking I/O.
	FAULT CONTAINMENT Limit blast radius of bugs, leaks, and memory corruption.

Pick the unit that matches your reliability, performance, and operability goals.

2 CORE IDEA

PROCESS: owns its virtual address space and resources.

THREADS: are execution streams within a process. They share the process's memory/resources, but each thread has its own stack, registers, and TLS.



3 INTERNALS

	IDs	PID (process ID) identifies a process. TID (thread ID) identifies a thread.
	Address Spaces	Each process has its own virtual address space and page tables.
	Memory Layout	Code, heap, globals, and mmap regions are shared by threads in a process. Each thread has a private stack, registers, and TLS.
	Scheduler	Kernel schedules threads (not processes) on available CPU cores.
	Kernel vs User Threads	Kernel threads are managed by OS. User-level threads (green threads/fibers) scheduled in user space by runtime.
	Creation	Process: <code>fork()</code> / <code>exec()</code> or <code>spawn</code> . Thread: <code>pthread_create()</code> , <code>_beginthreadex()</code> , <code>Thread.start()</code> , <code>std::thread</code> , etc.
	Context Switch	Save/restore registers, PC, stack pointer, and scheduler bookkeeping. Thread switch is cheaper than process switch.
	Fork & Copy-on-Write	<code>fork()</code> creates child with copy-on-write pages; cheap until pages are written.
	IPC	Pipes, sockets, msg queues, shm, files, signals, RPC, gRPC, etc.
	Signals	Delivered to process; directed to a thread (e.g., <code>pthread_kill</code>).
	Memory Model	Threads share memory; must synchronize to avoid data races (locks/atomics).

4 COMPLEXITY

This is not Big O; it's system-level trade-offs.

Aspect	Processes	Threads	Notes
Startup Cost	High	Low	Process needs address space setup; threads are cheap.
Context Switch	High	Low	More state (mmu, TLB, files).
Memory Overhead	High	Low	Separate address space, stacks, page tables, kernel structs.
Communication	Harder	Easier	IPC vs shared memory.
Isolation	Strong	Weak	Faults contained vs shared fate.
Synchronization	Less	More	Threads need locks/atomics.
Failure Blast Radius	Small	Large	Crash affects only itself vs all threads in process.
Cache/TLB Locality	Lower	Higher	Threads share address space.
Best For	CPU-bound Parallelism	I/O-bound Concurrency	Use cores vs hide latency.

CPU-bound work → processes/threads across cores.
I/O-heavy work → threads/async to keep one process busy.

5 TRADE-OFFS

PROCESSES	THREADS
✓ Strong isolation & security ✓ Crash containment ✓ Easier to reason/debug ✓ Easier restart and recovery	✓ Cheap to create ✓ Fast context switches ✓ Share memory/resources ✓ Easy communication
✗ Higher memory usage ✗ Slower startup ✗ IPC is harder/expensive ✗ Harder to share state	✗ Data races, deadlocks ✗ Bugs affect all threads ✗ Harder to debug ✗ Needs careful sync
WHEN TO USE	
Use PROCESSES when:	Isolation, security, multi-tenant, untrusted code, fault containment, infrequent communication.
Use THREADS when:	Low-latency communication, shared state, I/O concurrency, many concurrent tasks.
HYBRID PATTERNS	
Process-per-service + thread pools, or thread pools inside worker processes; async + threads + processes.	

6 DISCUSSION

- 1 When would you choose processes over threads (and vice versa)?
- 2 What happens during a context switch?
- 3 How does the GIL in CPython affect threading?
- 4 `fork()` vs `spawn()`: differences and implications?
- 5 How would you share data between processes?
- 6 What happens when a thread crashes? What about a process?
- 7 How big is a thread stack? How is it allocated?
- 8 What are user-level threads / fibers / green threads?
- 9 How does CPU affinity influence performance?
- 10 How do containers/namespaces relate to processes?
- 11 How would you limit runaway processes/threads?
- 12 How would you design a system for graceful shutdown and cancellation?

7 RELATED

	Scheduling Preemptive, time-slicing, priorities, run queues.
	Context Switching CPU state save/restore, TLB/cache effects.
	Virtual Memory Paging, page tables, copy-on-write, TLB.
	IPC Pipes, sockets, shared memory, message queues, RPC.
	Synchronization Mutexes, RW locks, semaphores, atomics, condition variables, barriers.
	Async & Coroutines Event loops, callbacks, futures/promises, structured concurrency.
	Containers & Namespaces PID, mount, network, IPC, user namespaces.
	Actors & Message Passing Isolate state by communicating, not sharing.

8 PRODUCTION EXAMPLES

	Web Servers	Nginx (multi-process workers), Apache prefork/event.
	JVM Servers	Threads handle requests; JVM manages threads, GC, and safepoints.
	Browser Architecture	Multi-process: UI, renderer (per tab/site), GPU, network, extensions—crashes stay contained.
	Databases	PostgreSQL: multi-process (backends) + threads. MySQL/InnoDB: threads (connection/workers).
	Job Workers	Celery/RQ/Sidekiq workers are processes; threads for concurrency within a worker.
	Media Pipelines	Separate processes for decode/encode; threads in stage for parallel filters.
	ML / Data Workloads	Data loaders (threads), compute (processes/GPU), model parallelism across devices/processes.
	Sandboxing	Run untrusted code in separate processes or containers.
	Microservices Sidecars	Sidecars (proxies, log shippers) are separate processes or containers.
	Node.js	Single-threaded event loop for I/O; worker_threads module for CPU-bound tasks.

9 INTERVIEW THINKING

- 1 Understand the workload
CPU-bound or I/O-bound? Latency vs throughput?
- 2 Isolation & security
Do you need fault containment or trust boundaries?
- 3 Sharing requirements
How much state needs to be shared?
- 4 Failure policy
What happens on crash? Restart strategy?
- 5 Runtime constraints
Language/runtime (GIL, green threads, actors, etc.).
- 6 Coordination model
Locks/atomics vs message passing/IPC?
- 7 Scale targets
Single host vs multi-host? #cores and memory?
- 8 Observability & ops
How will you monitor, debug, and troubleshoot?
- 9 Decide & iterate
Choose process/thread/async/hybrid and validate.

10 SENIOR NOTES

- Concurrency is not parallelism. Threads help concurrency; cores provide parallelism.
- Processes isolate faults, but distributed state still needs consistency, idempotency, and retries.
- Threads share heap, not stack; each has its own stack, registers, and TLS.
- Language runtime may constrain execution (e.g., GIL, STW GC, green threads). Don't assume unlimited parallelism.
- Measure before optimizing: context switches, run queue length, CPU, I/O wait, lock contention, oversubscription.
- Design for cancellation, deadlines, backpressure, and graceful shutdown.

11 MENTAL HOOK

Processes isolate worlds; threads share a world while taking separate execution paths.

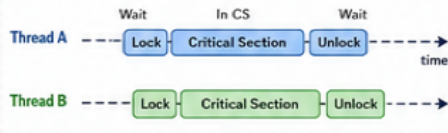
LOCKS & MUTEXES

Senior Software Engineer / Technical Lead Interview Prep (2-3 Minute Review)

1 WHY

- Preserve invariants over shared mutable state.
- Serialize critical sections to prevent race conditions.
- Establish synchronization & happens-before relationships between threads.
- Provide safety (mutual exclusion) and visibility (memory ordering).

CRITICAL-SECTION TIMELINE



Only one thread at a time is in the critical section.

2 CORE IDEA

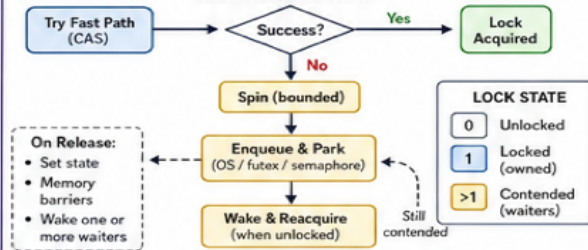
1. Acquire the lock.
 2. Enter exclusive critical section.
 3. Release the lock.
- A mutex has an owner (the holding thread).
 - Protect the data/invariant, not the code line.
 - Keep the critical section as small as is safely possible.
 - Use RAII / scoped locking to avoid leaks on exceptions/early returns.

RAII / SCOPED LOCKING (C++)

```
std::mutex m;  
  
void update(Account& a) {  
    std::lock_guard<std::mutex> lk(m); // acquires  
    a.balance += 100;  
} // lk goes out of scope → releases
```

3 INTERNALS

LOCK ACQUIRE PATH



LOCK STATE	
0	Unlocked
1	Locked (owned)
>1	Contended (waiters)

Concept	Notes
Fast Path	Atomic CAS to change state if unlocked. No syscalls.
Slow Path	Spin for a short time, then park via OS (futex/semaphore).
Wait Queue	Threads queue FIFO/LIFO (impl-dependent) while waiting.
Memory Model	Acquire on lock, Release on unlock → happens-before.
Uncontended	Very cheap (~O(1)): a few atomics.
Contended	Scheduler & OS involved → much heavier.
Variants	Recursive mutex, Read-Write lock, Spin lock, Fair locks.
Design Knobs	Lock granularity, striping, sharding, partitioning.
Hazard	Priority inversion → use priority inheritance/ceiling.

4 COMPLEXITY

Scenario	Acquire Cost	Notes
Uncontended	~O(1)	A few atomic ops (no blocking).
Contended (Active Wait)	Unbounded	Spinning wastes CPU, duration depends on CS length.
Contended (Blocked)	Unbounded	Sleep/wakeup + scheduler latency. Could be ms+.
Throughput	Varies	Impacted by contention, CS length, cache misses.
Latency	Varies	Cache-line bouncing, context switches add high tail latency.
Scaling	Limited by serialization	Amdahl's Law applies: speedup ≤ 1 / (S + P/N)



Main cost drivers: contention, critical-section length, cache-line bouncing, context switches, preemption, and NUMA effects.

5 TRADE-OFFS



What You Get

- Simple, strong mutual exclusion
- Easy to reason about
- Broadly understood & well-supported



What You Risk

- Deadlock (cycles in lock ordering)
- Convoying (thundering herd, cache ping-pong)
- Starvation (unfair locks)
- Priority inversion
- Reduced parallelism / scalability



When Not to Use Locks

- Immutable data or thread confinement
- Atomic operations (when sufficient)
- Message passing / actor model
- Lock-free data structures (only with expertise and proof of need)



Prefer the simplest correct approach. Reach for locks first. Optimize only with measurements.

6 DISCUSSION

- ? Mutex vs. semaphore—what's the difference?
- ? How does a monitor (lock + condition variable) work?
- ? When would you choose spin vs. block?
- ? When would you use a read-write lock?
- ? What are reentrant (recursive) locks? Pros/cons?
- ? How do locks provide memory visibility?
- ? How do you avoid deadlocks? Lock ordering?
- ? What is double-checked locking? Safe or not? (Why?)
- ? How do you ensure fairness? When does it matter?
- ? What is priority inversion and how do you handle it?

7 RELATED

🚶 Race Conditions	Concurrent access without proper synchronization.
🔗 Deadlock	Circular wait; needs prevention, avoidance, detection, or timeout.
⚙️ Atomics / CAS	Low-level primitives for lock-free algorithms and fast paths.
💬 Condition Variables	Block until a predicate becomes true; used with a lock.
🏠 Semaphores	Counting permits; broader than mutex (can signal).
📦 Memory Model	Acquire/Release, fences, happens-before guarantees.
👁️ Monitors	Encapsulate state + lock + condition variables.
📄 Transactions	ACID properties; isolation vs locking.

8 PRODUCTION USES

🗄️ Caches	Eviction lists, LRU updates, metadata.	🏠 Account Updates	Balances, transfers, ledgers.
🗄️ DB Buffers	Buffer pool, index structures, pages.	🔧 Allocator Metadata	Free lists, arenas, slabs.
📄 Logging	Append-only buffers, rotations, flushing.	👥 Connection Pools	Checkout/checkin, limits.
⚙️ Kernel Structures	Schedulers, run queues, filesystems, sockets.	★ Singleton Init	Lazy initialization of singletons.
📊 Metrics Aggregation	Counters, histograms, export snapshots.	🔒 File Locks (OS)	Inter-process locks; different from mutexes.

Use the right tool: in-process mutexes ≠ cross-process file locks.

9 INTERVIEW THINKING

- 1 State the invariant you must protect.
- 2 Identify shared data and who reads/writes it.
- 3 Define ownership: who can change what.
- 4 Choose the smallest safe critical section.
- 5 Establish lock ordering to avoid cycles.
- 6 For waits: use condition variables + loop on predicate (always re-check).
- 7 Handle failure & cancellation: release in finally/RAII.
- 8 Consider contention: measure, profile, and reason about hotspots.
- 9 Test with race detectors and stress tools.

10 SENIOR NOTES

- ✓ A lock protects an invariant, not a function or a line.
- ✓ Always release in finally/RAII—no exceptions.
- ✓ Never block on I/O or network while holding a lock without a clear, documented rationale.
- ✓ Consistent lock ordering prevents cycles and deadlocks.
- ✓ Read-write locks can underperform a mutex—measure.
- ✓ Correctness first; then measure and reduce contention.
- ✓ Lock-free is not wait-free; retries can still starve.
- ✓ Design for observability: expose contention metrics.
- ✓ Know your platform: memory model & primitives matter.



A mutex buys a simple invariant by making contenders wait at one door.



SEMAPHORES

Senior Software Engineer / Technical Lead Interview Cheatsheet

1. WHY

Coordinate limited concurrent access and signal availability/events.

- Resource pools (DB, connections, GPU)
- Bounded buffers and queues
- Flow control and backpressure
- Throttle concurrency and throughput
- Synchronize producers and consumers



2. CORE IDEA

Semaphore maintains an integer count of available permits.

wait() / P() / down(): acquire a permit.
Decrement count; if count < 0, block.
signal() / V() / up(): release a permit.
Increment count; if count ≤ 0, wake a waiter.

COUNTING SEMAPHORE (General)

Initial count = N (N ≥ 0)
Allows up to N concurrent acquirers.
Used for resource pools, flow control, buffers.

BINARY SEMAPHORE (Special case)

Initial count = 0 or 1
Acts like a gate or event (0 = closed, 1 = open).
Not required to express mutual exclusion.

No ownership requirement (unlike mutex): any thread can signal a semaphore.

3. INTERNALS



Atomic counter for permits (fast path): if count > 0 then decrement and proceed.



Wait queue when count ≤ 0: threads enqueue and park (block).



Wake policy: signal increments count; if waiters exist, wake one (or more by policy).



Memory ordering: acquire on wait success; release on signal to publish prior writes.



Bounded maximum: count has a safe upper bound to prevent overflow.



Fairness: FIFO (common) vs barging: impacts starvation and throughput.



Cancellation / timeouts: wait may return on timeout or cancellation; handle cleanup.



Variants: unnamed (in-process), named (semaphores in OS), process-shared.



Condition patterns: use semaphore as an event, gate, or to pass tokens.

4. COMPLEXITY

Aspect	Notes
Uncontended acquire/release	O(1)
Contended wait / wake	Unbounded (depends on wait time)
Throughput	Bounded by permits × service rate
Little's Law link	$L = \lambda \times W$ (concurrency × throughput × latency)
Memory	O(# of waiting threads) + semaphore object
Costs	Context switches, cache misses, convoying, kernel transitions under contention

Permit-counter state machine

ACQUIRE (wait/P)	RELEASE (signal/V)
count > 0 → count--; proceed	count < 0 → count++, wake one
count ≤ 0 → count--, block	count ≥ 0 → count++



5. TRADE-OFFS

PROS

- Flexible: counting, signaling, backpressure.
- Simple mental model; minimal primitives.
- Controls concurrency and smooths load.
- Useful across in-process and OS boundaries.

CONS / RISKS

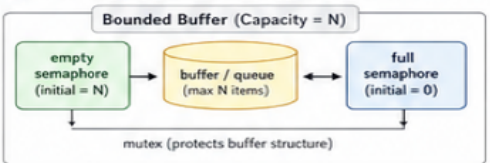
- Permit leaks (not released) → capacity loss.
- Over-release → count inflation, breaks limits.
- Starvation / unfairness under contention.
- Harder to reason about invariants than mutex.
- Not automatic mutual exclusion or ownership.

WHEN NOT TO USE

- Protecting a single invariant / critical section → use mutex.
- Waiting for a condition/predicate → use condition variable.
- Message passing / data handoff → use channel/queue.

6. DISCUSSION (Interview Prompts)

- How would you explain the difference between a mutex and a semaphore?
- What is a binary semaphore and when is it useful?
- How do you choose the initial count?
- Design a bounded buffer with two semaphores + mutex. Draw the flow.
- How do you ensure fairness? FIFO vs barging trade-offs?
- How do you detect and prevent permit leaks?
- How do you handle cancellation and timeouts safely?
- How would you implement a rate limiter with a semaphore?
- How would a distributed semaphore work? What are the challenges?
- What metrics and alerts would you add in production?



7. RELATED CONCEPTS

Mutexes	Exclusive access with ownership.
Condition Variables	Wait for predicates, not just slots.
Producer-Consumer	Classic pattern using semaphores.
Bounded Queues	Built on semaphores + mutex.
Thread Pools	Semaphore limits concurrent workers.
Channels / Message Queues	Pass data; often clearer semantics.
Barriers / Latches	Phase synchronization primitives.
Rate Limiting / Token Bucket	Semaphores model available tokens.

8. PRODUCTION USE CASES

DB Connection Pools	Limit concurrent DB connections.
HTTP / API Concurrency Caps	Limit in-flight requests to upstream.
Worker Slots / Job Executors	Limit concurrent background jobs.
Bounded Uploads / Downloads	Throttle simultaneous transfers.
GPU / Device Resources	Limit kernels / streams / contexts.
File Descriptors / Handles	Limit open files, sockets, etc.
Producer-Consumer Buffers	Limit buffer occupancy.
API Fan-out / Outbound Calls	Limit concurrent outbound calls.
Deployment Gates	Rollout slots, migration windows.

9. INTERVIEW THINKING FRAMEWORK

- Define the resource or event to control.
- Determine capacity (permits) and units.
- Decide acquire (wait) and release (signal) points; who owns release responsibility.
- Handle failures, cancellation, and timeouts; guarantee release with structured cleanup.
- Set maximum bound; consider dynamic resizing?
- Consider fairness, queueing policy, and starvation risks.
- Combine with mutex/condition variable to protect any shared data structure or invariants.
- Add observability: metrics, saturation, wait time, errors.
- Validate with stress tests: load, spikes, faults, timeouts.

Key Metrics

- Permits in use
- Available permits
- Waiters (queue length)
- Wait time (p50/p95/p99)
- Acquire rate / Release rate
- Saturation (% of capacity)

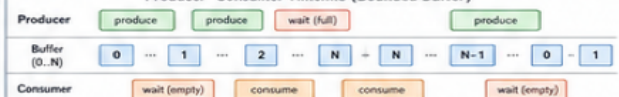
10. SENIOR NOTES

- Semaphores represent capacity, not data. They gate work.
- Release exactly once per acquire; use defer/finally pattern.
- A semaphore alone does not protect collection invariants; use a mutex/condition variable around the structure.
- Limits shift queues: when you throttle here, backlog moves upstream or downstream—understand the system.
- Distributed semaphores need leases, expirations, and fencing tokens to avoid split-brain and stale holders.
- Observe and tune: long wait times or high saturation indicate bottlenecks or mis-sized limits.

11. MENTAL HOOK

A semaphore turns finite capacity into permits that concurrent work must borrow and return.

Producer-Consumer Timeline (Bounded Buffer)



Permit Counter Example (Initial count = 2)

Step	0	1	2	3	4	5	6	7	8	9	10
Action	-	P	P	P	V	P	V	V	P	P	V
Count	2	1	0	-1	0	-1	0	1	0	-1	0
Waiters	0	0	0	1	0	1	0	0	0	1	0

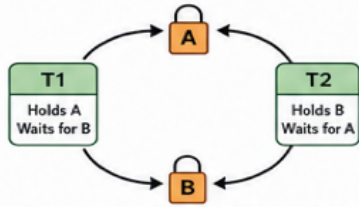
P = wait/acquire V = signal/release

DEADLOCK

Senior Software Engineer / Technical Lead Interview Cheat Sheet

1. WHY

- Concurrent actors can wait forever despite healthy components.
- Availability failure from circular resource dependency.



System is "healthy" but progress is impossible.

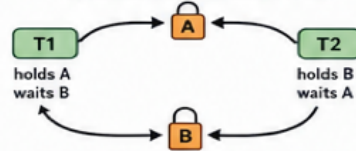
2. CORE IDEA

Deadlock = cycle of blocked work: each participant holds a resource and waits for the next.

Coffman Conditions (all must hold):

- Mutual Exclusion** – resource cannot be shared.
- Hold and Wait** – holds at least one resource while waiting for others.
- No Preemption** – resources cannot be forcibly taken; only released.
- Circular Wait** – a cycle of waiting exists.

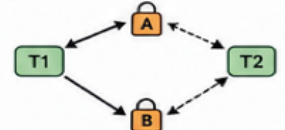
Cycle Example (2 actors, 2 locks)



3. INTERNALS

Lock / Resource Allocation Graph

- Process/Thread (green box)
- Resource (orange square)
- holds (allocated)
- > waits (request)



Wait Queues

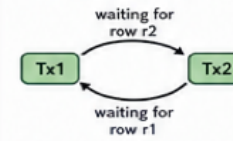
Lock A: [T2, T3, ...]
Lock B: [T1, T4, ...]

Queues blocked on resource.

Two-Lock ABBA Trace

Step	T1	T2
1	lock(A)	
2		lock(B)
3	lock(B) ← waits	
4		lock(A) ← waits

Database Wait-For Graph



Key Internals

- Scheduler cannot fix a logical cycle.
- Distributed systems add latency, partitions, and partial failure.
- Leases and timeouts change failure mode, not correctness.
- Livelock: active but no progress.
- Starvation: indefinite delay, not cycle.

4. COMPLEXITY



Detection (Wait-For Graph Cycle):
DFS in $O(V + E)$
 V = processes, E = edges (waits)



Prevention: runtime cost
Global ordering, extra bookkeeping, try-lock loops, or transactions add latency and reduce concurrency.



Timeouts: bound wait
Detect after T seconds → break wait.
False aborts possible under load.



Recovery: victim + rollback
Abort a victim, release resources, restart. Choose victim by cost (work done), priority, or age.



Deadlock probability/latency is not a Big O quantity: depends on workload, contention, scheduling, and topology.

5. TRADE-OFFS

Strategy	How it works	Pros	Cons
Global Lock Ordering	All code acquires locks in the same order.	Simple, no cycles.	Hard to retrofit; limits flexibility.
Acquire-All (Static Set)	Collect all needed locks first, then proceed.	No hold-and-wait.	Need full set upfront; may block longer.
Try-Lock + Backoff	Try to acquire; on failure, release and retry with backoff.	Good concurrency.	Retries increase latency; tuning needed.
Timeouts	Abort wait after T seconds.	Simple recovery.	False aborts; doesn't prove absence.
Preemption / Transactions	Abort/rollback a holder to release resources.	Breaks hold-and-wait / no-preemption.	Costly rollbacks; complex semantics.

Avoidance (Banker's Algorithm): grant only if system remains in a safe state. Rarely used in general-purpose production systems due to overhead and incomplete information.

6. DISCUSSION

- ? What are the four Coffman conditions?
- ? How do prevention, avoidance, and detection/recovery differ? Which would you use and why?
- ? How would you enforce lock ordering in a large codebase?
- ? Is a timeout sufficient to prevent deadlock? When is it appropriate?
- ? How do databases detect and resolve deadlocks?
- ? What is livelock vs deadlock? How are they different?
- ? How do distributed locks, leases, and fencing tokens help? What are their pitfalls?
- ? Given a thread dump, how do you diagnose a deadlock?

7. RELATED

- Mutexes / Monitors / Semaphores
- Race Conditions (safety) vs Deadlock (liveness)
- Starvation (indefinite delay)
- Livelock (active but no progress)
- Wait-For Graph (WFG)
- Transactions & ACID
- Two-Phase Locking (2PL)
- Leases, TTL, Fencing Tokens

8. PRODUCTION EXAMPLES

- Database transactions: row/page/metadata locks; deadlock victim and automatic retry.
- Application code: two methods lock resources in different orders (ABBA).
- Thread pools: parent waits on child future holding a lock; children need that lock.
- Connection pool exhaustion: threads hold DB connections while waiting for others → cycle across pools.
- Distributed services: service A holds lease, calls B; B calls A back needing A's lease.
- Filesystem operations: multiple file/dir locks across paths.
- Deployment/orchestration: rolling updates waiting on resources held by old pods.
- UI thread joins: blocking the UI while holding a lock needed by the UI thread.

9. INTERVIEW THINKING

- Inventory resources (locks, connections, queues, leases).
- Map ownership (who holds) and waits (who wants what).
- Draw the wait-for graph.
- Identify cycles.
- Choose a condition to break (prefer prevention).
- Apply strategy: ordering, timeouts, or rollback/preemption.
- Add observability: metrics, logs, lock owners, wait times.
- Reproduce and stress test (load, chaos, fault injection).
- Define recovery contract: retries, idempotency, SLA.

10. SENIOR NOTES

- Consistent global order is the simplest, most reliable prevention.
- Timeouts recover symptoms, not prove absence.
- Never wait on external I/O while holding a scarce lock without a design (can cascade).
- Thread dumps need ownership metadata (lock owner thread, hold time, waiters).
- DB deadlocks are expected; handle with retry + backoff.
- Distributed leases require fencing to prevent split-brain incorrectness.
- Measure lock hold times and contention hot spots.
- Design APIs to minimize lock scope and callouts.

11. MENTAL HOOK

Deadlock is a cycle where every participant owns the key another participant needs.



Common Wait-For Graph Patterns

Pattern	Meaning
	Some waiting, but progress is possible.
	Deadlock present.
	Thread waiting on a resource it already holds (bug or recursive lock misuse).

Quick Detection Signals

- Threads in WAITING / BLOCKED for long durations
- No CPU progress + stable stack traces
- Increasing queue lengths
- DB deadlock errors
- Rising lock wait time percentiles
- Circular dependencies in dependency graphs

Quick Mitigations

Retry + Backoff	Timeout	Reduce Lock Scope	Lock Ordering	Abort Victim

RACE CONDITIONS

Senior Software Engineer / Technical Lead Interview Prep – 2–3 Minute Engineering Map

1 WHY

Outcome depends on uncontrolled timing/interleaving of concurrent actions. Produces rare, non-deterministic correctness and security failures.

Why it hurts

- Heisenbugs: rare, timing-dependent
- Data corruption, invariant violation
- Security: TOCTOU, privilege bypass
- Hard to reproduce, easy to ship
- Impact scales with concurrency & scale

Examples of failures

	Lost update → wrong balance
	TOCTOU → auth bypass
	Oversell inventory
	Duplicate job execution

2 CORE IDEA

Two or more actors access the same shared state concurrently, at least one writes, without sufficient synchronization. If the outcome depends on the timing/interleaving, we have a race condition.

Data race (language-dependent)

Conflicting accesses (at least one write) to the same memory location, not ordered by happens-before. (C/C++/Rust: UB. Java: race - may see stale values.)

Mini example: Lost Update

Initial: x = 0			
Thread A		Time	Thread B
r1 = x	1		r1 = x
r1 = 0	2		r1 = 0
r1 = r1 + 1 (→1)	3		r1 = r1 + 1 (→1)
x = r1 (store 1)	4		x = r1 (store 1)
Final: x = 1 (should be 2)			

Possible interleavings (partial order)



4 COMPLEXITY

Not about Big O—about interleaving explosion.

- State space grows combinatorially with threads and shared variables.
- Even “correct on my machine” rarely generalizes.

Sources of cost

	Fixes can serialize	Locks/critical sections reduce parallelism & throughput.
	Atoms can spin	CAS loops may retry under contention (livelock-like behavior).
	Detector overhead	Race detectors instrument memory accesses; slow & memory hungry.
	Low probability ≠ low severity	Rare races still cause outages, data loss, or security issues.

P(rare) reduces frequency, not impact.

5 TRADE-OFFS

Approach	Pros	Cons	Best When
Locks	Simple, familiar, composable	Blocking, priority inversion, deadlock	Low contention, complex invariants
RW Locks	More read concurrency	Writer starvation possible	Read-heavy + tolerate risk
Atoms	Lock-free, fast in uncontended cases	Limited to single locations; harder semantics	Counters, flags, state machines
Immutability	No races by design	Copying cost, GC pressure	Read-mostly, snapshots
Confinement	Simple mental model	Ownership transfer is work	Actor model, message queues
Message Passing	Loosely coupled, scales	Latency, queue management	Distributed components
Transactions	Atomic multi-variable updates	Overhead, aborts, contention	Strong invariants across items

Atoms are insufficient when:

- Invariant spans multiple variables (e.g., $a+b \leq N$)
- You need to read-modify-update multiple locations atomically
- You need to condition on a set of values and act as one unit

3 INTERNALS

Read-Modify-Write is not atomic

x++ expands to:



Why interleavings occur

- CPU caches & coherence
- Store buffers / write queues
- Out-of-order execution
- Compiler reordering
- Preemption / context switch

Memory Model Essentials

Happens-Before (HB)

A HB B ⇒ A's effects visible to B.

Built from:

- Program order (within thread)
- Synchronizes-with (e.g., lock release → acquire, atomic ops)
- Transitive closure

Atomics & Memory Order (examples)

Order	Guarantees (roughly)
Relaxed	Atomicity only
Acquire	Subsequent reads/writes not moved before
Release	Prior reads/writes not moved after
Acq-Rel	Both acquire & release
Seq-Cst	Total order; easiest to reason

Three properties (do not conflate)

Visibility	Atomicity	Ordering
When a write becomes observable to others.	All-or-nothing w.r.t. interleaving.	Which operations appear before/after others.

TOCTOU (Time-Of-Check-To-Time-Of-Use)

Logical race between a check and the subsequent use.
Example: check file perms → open file

False Sharing (not a race)

Threads modify different variables on same cache line → performance degradation, not incorrectness.

6 DISCUSSION

- ? Race condition vs data race?
All data races are race conditions; not all race conditions are (e.g., TOCTOU, logical races).
- ? Is volatile enough?
Volatile/atomic gives visibility (ordering constraints), not atomicity of compound actions.
- ? Is increment atomic?
Not generally. $x++$ is read-modify-write; use atomic `fetch_add` or lock.
- ? What establishes happens-before?
Program order, lock/unlock, atomic ops, fences, thread start/join, etc.
- ? Double-checked locking safe?
Only with proper memory fences/volatile on the published reference.
- ? What is ABA?
CAS sees $A \rightarrow B \rightarrow A$ as unchanged. Mitigate with version stamps or hazard pointers.
- ? TOCTOU examples?
File check then open, access check then use, existence check then create.
- ? How to detect/test races?
Static analysis, dynamic race detectors, logging, fuzzing, stress, chaos, model checking.
- ? Deterministic reproduction?
Control scheduler, seed, virtual time, record & replay, systematic concurrency testing.
- ? Distributed races?
Concurrency across nodes: use idempotency, version checks, consensus/leases.

7 RELATED

	Mutexes / Locks	Mutual exclusion, critical sections.
	Memory Model	JMM, C++ memory model, .NET CLJ, Rust.
	Atoms / CAS	Compare-And-Swap, LL/SC, fetch_*, fences.
	Deadlock	Circular wait, prevention & detection.
	Immutability	Persistent data structures, copy-on-write.
	Actors / Confinement	Single-threaded ownership per actor.
	Transactions	ACID, isolation levels, optimistic vs pessimistic.
	Linearizability	Stronger guarantee than sequential consistency.

8 PRODUCTION SCENARIOS

	Account balance updates (lost update / write skew)		Leader election / lease races
	Inventory reservation / oversell		Duplicate job processing / at-least-once
	Cache population / thundering herd		UI state races (async events)
	Lazy initialization (singletons)		Database write skew (snapshot isolation)
	File creation / auth TOCTOU		Distributed idempotency (dedupe keys)
	Counters / metrics accuracy		

Symptoms: intermittent failures, impossible states, missing or duplicate work.

9 INTERVIEW THINKING (A PLAYBOOK)

- 1 Define the invariant / correctness property.
- 2 Identify all shared state & who accesses it.
- 3 Enumerate interleavings that break the invariant.
- 4 Choose an approach: ownership, sync, or redesign.
- 5 Prove safety: establish happens-before or atomic transition for the invariant.
- 6 Handle failure: timeouts, retries, cancellations, idempotency.
- 7 Instrument: logs, metrics, invariants, assertions.
- 8 Stress & verify: race detector, fuzz, load, chaos.

Think in terms of: who owns the state, who can change it, and what orders those changes.

10 SENIOR NOTES

- ✓ Thread-safe fields/collections ≠ thread-safe workflow. Invariants span operations, not data types.
- ✓ Volatile gives visibility and ordering, not atomicity of compound actions.
- ✓ Atomics require memory-order reasoning: defaulting to `seq_cst` is safe but may be slower.
- ✓ Race detectors find executions, not proofs; absence of a report ≠ absence of races.
- ✓ Distributed systems widen the surface: network, clocks, partitions. Design with idempotency, version checks (ETags, revisions), and reconciliation.
- ✓ Prefer reducing shared mutable state and increasing ownership; good design beats clever locking.

11 MENTAL HOOK



A race condition lets timing become an undocumented input to correctness.

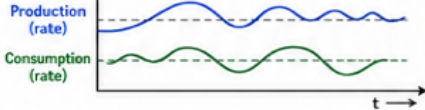
PRODUCER-CONSUMER

Engineering Map for Senior Software Engineer / Technical Lead Interview (2-3 Minute Review)

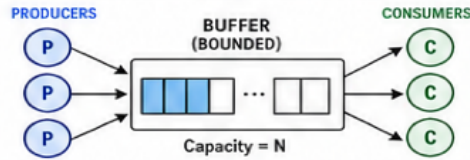
1. WHY



- Decouple work generation from processing rate
- Absorb bursts and smooth variability
- Enable parallel pipelines and better resource utilization
- Enforce backpressure to protect downstream systems



2. CORE IDEA



Producers enqueue items, consumers dequeue. Shared bounded buffer between them.

Correctness: no loss, duplication, corruption.

Liveness: progress and clean shutdown.

3. INTERNALS

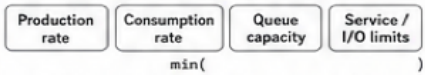
- Circular queue (ring buffer) head/tail wrap around



- Mutex protects queue structure (head, tail, size, buf[])
- Empty / Full semaphores or Condition Variables
 - wait in a while loop on predicate
 - while (!can_proceed) wait();
- MPMC (multiple producers, multiple consumers)
- Close / end-of-stream via sentinel or explicit flag
 - e.g., POISON_PILL or close()
- Ownership transfer on enqueue (move semantics)
- Batching to reduce lock/notify overhead
- Work stealing queues as alternative model
 - better cache locality for workers

4. COMPLEXITY

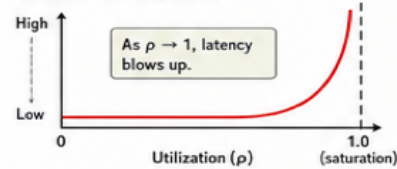
- Enqueue / Dequeue: $O(1)$ amortized
- Memory: $O(\text{capacity})$
- Throughput is bounded by the minimum of:



- Little's Law relates average items in system (L), arrival rate (λ), and average time in system (W):

$$L = \lambda W$$
$$W = L / \lambda$$
$$L = \lambda \times W$$

- Queueing latency grows non-linearly as utilization $\rightarrow 1$ (saturation).



5. TRADE-OFFS

Benefit	Cost / Risk
Smoothing bursts, decoupling rates	Adds latency (queueing delay)
Parallelism & better resource utilization	More memory, context switching
Backpressure protects slow downstream	Upstream blocking or dropped work
Bounded queue gives predictable behavior	Must choose policy: block / drop / spill
Pipelines improve throughput	Ordering complexity, head-of-line blocking
Isolation of failures between stages	Error handling & retries are harder

Bounded Queue Overload Policies

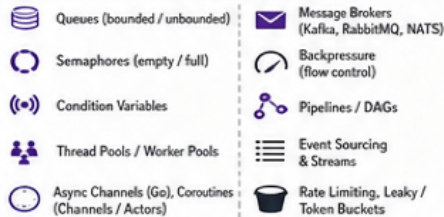
BLOCK	Producer waits when full (backpressure).
DROP	Reject new item (lossy; needs metrics).
SPILL	Write to disk / external store (higher latency).

Unbounded queues risk unbounded memory $\rightarrow$ OOM, GC thrash, tail latency, system collapse.

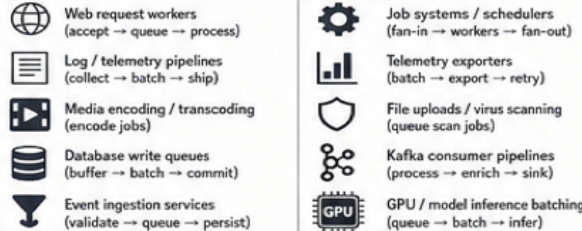
6. DISCUSSION (Interview Prompts)

- How would you implement a bounded buffer?
- Why use while (not if) around wait predicate?
- How does your design handle multiple producers and multiple consumers?
- How do you handle shutdown and draining?
- Poison pill pitfalls? (multiple consumers, re-queuing, ordering, early termination)
- How do you preserve ordering (if required)?
- How do you apply backpressure upstream?
- What happens when a consumer is much slower than producers?
- How do you provide at-least-once processing?
- Would you use a lock-free queue? Why or why not? When does it help? What are the trade-offs?

7. RELATED



8. PRODUCTION USE CASES



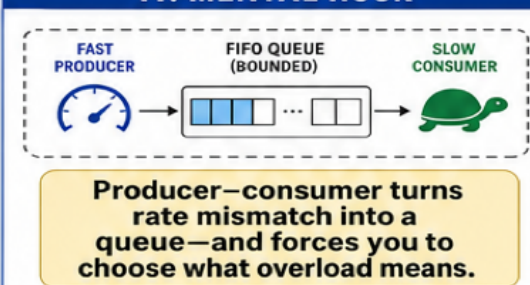
9. INTERVIEW THINKING (Design Checklist)

- Define item contract & ownership (who owns what, when).
- Understand rates: production, consumption, variability.
- Choose capacity bound using SLOs & memory budget.
- Decide overload policy: block / drop / spill (and signal).
- Choose synchronization: mutex + CV / semaphores / lock-free / channels.
- Define ordering guarantees (FIFO? per-key? none?).
- Define ownership transfer & lifetime (copy vs move).
- Failure & retry: idempotency, DLQ, retries, timeouts.
- Shutdown protocol: stop intake, drain, flush, ack, close.
- Observability: depth, age, rates, errors, saturation alerts.

10. SENIOR NOTES

- Queues move waiting, they don't remove it.
- Capacity is an SLO decision, not a magic number.
- Define ownership & immutability boundaries immediately after enqueue.
- Shutdown is a protocol, not an afterthought.
- Batch wisely: bigger batches $\uparrow$ throughput but $\uparrow$ latency and tail risk.
- Distributed producer-consumer adds delivery semantics, ordering, poison messages, and replay.
- Monitor depth, item age, throughput, error rate, retries, and saturation to act before it hurts.

11. MENTAL HOOK



Bounded Buffer with Semaphores (Pseudo)

```
Enqueue(item):
    wait(empty);
    lock(m);
    buf[tail] = item; tail = (tail + 1) % N;
    unlock(m);
    signal(full);

Dequeue():
    wait(full);
    lock(m);
    item = buf[head]; head = (head + 1) % N;
    unlock(m);
    signal(empty);
    return item;
```

With Condition Variables (Pseudo)

```
Enqueue(item):
    lock(m);
    while (size == N) not_full.wait(m);
    push(item);
    not_empty.notify_one(m);
    unlock(m);

Dequeue():
    lock(m);
    while (size == 0) not_empty.wait(m);
    item = pop();
    not_full.notify_one(m);
    unlock(m);
    return item;
```

Queue Metrics to Track

- Depth (current, avg, p95/p99)
- Item Age (oldest / p95 age)
- Enqueue / Dequeue rates
- Drop / Reject / Spill counts
- Error / Retry rate
- Saturation (% time full / near full)

Common Policies

- Backpressure: Block
- Lossy: Drop
- Durable Buffer: Spill
- Retry: Exponential + Jitter
- Idempotency: Keys / Dedup
- DLQ: Dead Letter Queue

THREAD POOLS

Senior Software Engineer / Technical Lead Interview Prep – 2–3 Minute Review

1 WHY

- **Reuse workers:** avoid expensive thread creation/destruction.
- **Bound concurrency:** cap memory, CPU, DB connections, etc.
- **Amortize cost:** warm threads, caches, JIT, TLS reuse.
- **Schedule many tasks safely** and predictably.

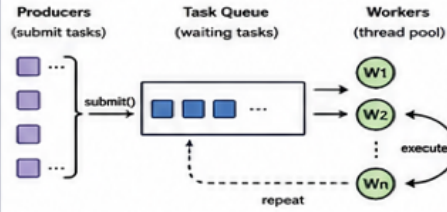
With Thread Pool

- ✓ Lower latency (no per-task thread creation)
- ✓ Bounded resources
- ✓ Better throughput under load
- ✓ Backpressure via queue policies

Without Thread Pool

- ❌ High creation/destruction overhead
- ❌ Unbounded threads → OOM / thrash
- ❌ Poor locality & cache churn
- ❌ Collapse under load

2 CORE IDEA



Tasks are submitted to a queue. Worker threads pull tasks, execute them, then repeat. Pool size and queue policy control throughput and backpressure.

3 INTERNALS

Task queue	Holds tasks waiting for execution.
Worker lifecycle	Create → start → run → idle → terminate.
Core / Max threads	core: kept alive; max: upper bound.
KeepAlive (idle timeout)	Idle > keepAlive → terminate (if > core).
Work stealing	Idle worker steals from others' queues.
Local / Global queues	Each worker may have local deque; plus global queue for submissions.
Rejection policies	What happens when queue is full and max threads reached.
Futures / Promises	Represent pending results, get()/await().
Exception handling	Capture & propagate to Future/handler; don't lose.
Thread-local leakage	ThreadLocals persist across tasks; clear after use.
Graceful shutdown	shutdown() → stop accepting new work, drain queue, finish tasks.
Drain / Cancel	shutdownNow() → cancel queued and interrupt running tasks.

4 COMPLEXITY

- **Submission:** $O(1)$ typical (enqueue + signal).
- **Queue ops:** $O(1)$ amortized; contention on lock/CAS and wakeups.
- **Memory:** $O(|\text{queue}| + \text{threads} \times \text{stack} + \text{overhead})$.
- **Throughput** bounded by $\min(\text{workers} \times \text{service_rate}, \text{resource limits})$.
- CPU-bound pool size $\approx \#cores (\pm 1)$ for peak throughput.
- I/O-bound pool size must be larger; depends on blocking ratio (see Little's Law).

Little's Law

$$L = \lambda W$$

L = avg # items in system (queue + running)

λ = throughput (tasks/sec)

W = avg time in system (sec)

⇒ L informs queue sizing and required workers.

I/O Pool Sizing Heuristic

$$N_{\text{threads}} \approx N_{\text{cores}} \times \left(1 + \frac{W_{\text{wait}}}{W_{\text{compute}}}\right)$$

W_{wait} = time waiting (I/O, locks, network)

W_{compute} = time doing CPU work

5 TRADE-OFFS

Benefits

- ✓ Lower overhead & better latency
- ✓ Bounded resource usage
- ✓ Smoother throughput
- ✓ Backpressure via queue policies

Risks / Costs

- ⚠ Queuing latency (added wait)
- ⚠ Starvation / unfairness
- ⚠ Deadlocks from nested waits
- ⚠ Context switching & CPU overhead
- ⚠ Hidden failures (lost exceptions, silent rejections)

Queue Capacity Choices

Bounded Queue	Unbounded Queue
+ Predictable memory	+ Never rejects (by default)
+ Backpressure	– Unbounded memory
+ Failure fast (rejection)	– Long tail latency
– Possible rejections	– Risk of OOM & collapse
– Requires sizing	

Treat queue capacity as part of the design.

6 DISCUSSION (Interview Prompts)

1. How would you size a thread pool for this workload?
2. Is the workload CPU-bound, I/O-bound, or mixed?
3. Fixed size, cached (unbounded), or work-stealing pool?
4. Which queue would you choose and why?
5. What rejection policy makes sense here?
6. How do you handle task exceptions?
7. How do you avoid thread starvation / deadlocks?
8. What if tasks block on I/O or other services?
9. How would you implement graceful shutdown?
10. How would you detect and handle overload?
11. How would you isolate different workloads?
12. What metrics and alerts would you use?
13. How would you propagate context (trace/user)?

7 RELATED

Producer-Consumer	Producers submit, workers consume.
Queues	BlockingQueue, SPSC/MPSC, Disruptor.
Executors / Futures	ExecutorService, CompletableFuture.
Async Programming	Callbacks, Promises, async/await.
Semaphores / Permits	Bound concurrency, limit resources.
Scheduling	Cron, delayed tasks, rate limiting.
Work Stealing	ForkJoinPool, deques, steal for balance.
Backpressure	Bound queues, signals, load shedding.

8 PRODUCTION USE CASES

Web request handling Handle HTTP requests concurrently.	Background jobs Email, notifications, reporting.
Database work Pool DB calls, limit connections.	Image processing Resize, transcode, apply filters.
Build systems Compile/link/test in parallel.	Fork-Join parallelism Divide & conquer CPU workloads.
RPC / Service clients High concurrency outbound calls.	Batch ingestion Process large datasets in parallel.
Scheduled tasks Periodic jobs, maintenance.	Virtual threads (note) Different model (Project Loom); see notes.

Match pool characteristics to workload and resource constraints.

9 INTERVIEW THINKING ROADMAP

1. Understand workload: concurrency, latency, blocking ratio.
2. Identify resource bottlenecks (CPU, I/O, DB, memory).
3. Define pool type, size range, and queue capacity.
4. Choose rejection policy & backpressure strategy.
5. Isolate workloads (by QoS, blocking, failure domain).
6. Handle cancellation, timeouts, deadlines.
7. Propagate context (trace IDs, security, locales).
8. Instrument: metrics, tracing, logs, alerts.
9. Load test, tune, and validate in production-like load.

10 SENIOR NOTES

- ★ Queue capacity is part of pool design; it shapes latency, memory, and failure modes.
- ★ More threads can REDUCE throughput (context switches, lock contention, cache misses).
- ★ Never let tasks wait synchronously on the same saturated pool without analysis.
- ★ Separate workloads by blocking behavior and failure profile.
- ★ Propagate context carefully; clear ThreadLocals and MDC after tasks.
- ★ Observe: queue depth, queue age, active threads, saturation, rejections, errors.
- ★ Design for graceful degradation: shed load early, protect critical paths.
- ★ Review defaults of your platform; know what the executor actually does.

11 MENTAL HOOK

A thread pool turns threads into a budget and tasks into a queueing problem.

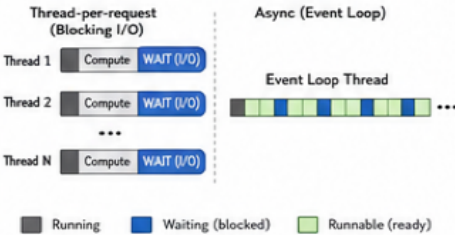
ASYNC PROGRAMMING

Senior Software Engineer / Technical Lead Interview Prep (2-3 Minute Review)

1 WHY

- Keep threads productive while operations wait.
- High-concurrency I/O without one blocked thread per request.
- Better resource utilization and scalability.
- Structured latency control and cancellation.
- Compose complex workflows without callback pyramids or thread explosion.

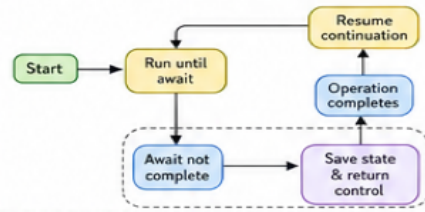
THREAD-PER-REQUEST vs ASYNC



2 CORE IDEA

- An async function runs until it hits an `await`.
- If the awaited operation is not complete, the function saves its continuation (state) and returns control to the runtime.
- When the operation completes, the runtime schedules the continuation to resume.
- Concurrency (overlap of waits), not necessarily parallelism (simultaneous execution).

LIFE OF AN ASYNC FUNCTION



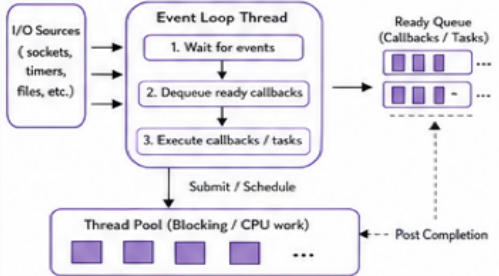
TIMELINE EXAMPLE

Time →	t0	t1	t2	t3	t4	t5
Async Function	Run	await	—	Resume	await	Resume
I/O Operation	Start	In flight	Complete	Start	In flight	Complete
Thread (Event Loop)	Runs others... (thread is free)					

3 INTERNALS

- Compiler transforms async function into a state machine (continuations).
- Futures / Promises / Tasks represent results.
- Runtime: Event Loop (Reactor) or Proactor.
- Readiness / Completion queues drive scheduling.
- Synchronization Context / Executor decides where continuations run.
- Thread pool for blocking or CPU work.
- Context propagation: call context, context vars, tracing, security.
- Structured concurrency scopes lifetime, cancellation, and error propagation.

ARCHITECTURE: EVENT LOOP (REACTOR)



4 COMPLEXITY

- Same algorithmic complexity (Big O) as sync code.
- Per-task state: O(1)-ish (state machine + handles).
- Event-loop scheduling & queueing add overhead.
- Memory: O(number of in-flight tasks + buffers).
- Throughput bounded by downstream resources (DB, network, disk, rate limits).
- Too much concurrency → saturation, timeouts, and retries amplify load.
- Async removes blocked threads, not work.

RESOURCE VIEW

Factor	Effect
In-flight tasks	↑ memory, queues, context switches
Event loop latency	↑ if callbacks block or CPU heavy
Downstream limits	Cap throughput (DB, API, I/O)
Thread pool size	Too small → starvation; too big → context switch / thrash
Backpressure	Prevents overload, stabilizes latency

LATENCY COMPONENTS



5 TRADE-OFFS

GOOD TRADE-OFFS

- Scalability with I/O-bound workloads
- Better resource utilization
- Composability (fan-out, pipelines)
- Cancellation & timeouts are first-class
- Lower memory footprint vs many threads

COSTS / RISKS

- APIs become "colored" (async all the way)
- Hidden scheduling → surprising behavior
- Harder debugging & stack traces
- Race conditions still possible
- Accidental blocking freezes event loop
- Fan-out overload → downstream collapse

WHEN NOT TO USE

- Simple sequential scripts / one-off tasks
- CPU-bound work without parallel executor
- 3rd-party libraries lack async support
- Ultra-low latency in-process CPU work

6 DISCUSSION (POSSIBLE INTERVIEW QUESTIONS)

- Async vs Threads: trade-offs and when to use each?
- What does 'await' actually do under the hood?
- What happens if the event loop is blocked?
- How do you run CPU-bound work in an async app?
- How can deadlocks occur with sync-over-async?
- How does cancellation work? What's cooperative cancellation?
- How are exceptions propagated across awaits?
- How does 'gather' / 'join_all' / 'Task.WhenAll' work? Fail-fast vs collect-all?
- How do you handle backpressure effectively?
- How do you propagate context / context variables?
- How do timeouts differ from cancellation?
- What is the cost of creating a task?
- How do you avoid accidental blocking calls?
- How do you test async code deterministically?

7 RELATED CONCEPTS

Concept	Purpose
Future / Promise / Task	Represents a result that will be available later
Callback	Function invoked on completion
Coroutine / async fn	Syntax for async state machine
Event Loop	Drives readiness & callbacks
Thread Pool / Executor	Runs blocking or CPU tasks
Reactive Streams	Push-based, backpressure-aware data flows
Channels / Queues	Async message passing
Structured Concurrency	Scopes for lifetime, cancel, errors

8 PRODUCTION USE CASES

Web Servers High-concurrency HTTP handling	RPC Fan-out Parallel downstream calls	DB / Network Clients Async drivers, connection pooling
File I/O Non-blocking file operations	UI Applications Responsive UI, no freezes	Message Consumers High-throughput stream processing
Crawlers / Scrapers Many small I/O requests	Serverless Handlers Handle many concurrent invocations	Streaming Pipelines Real-time data processing
Mobile Apps Battery-friendly, responsive network & I/O		

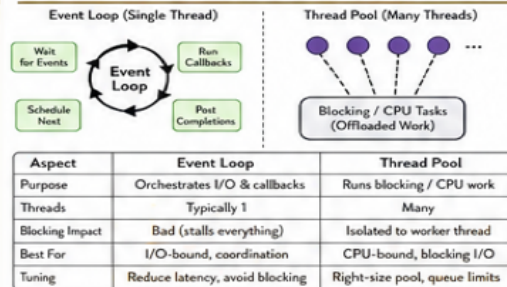
9 INTERVIEW THINKING CHECKLIST

- Identify what waits (I/O, timers, locks, rate limits).
- Use async-capable dependencies end-to-end.
- Determine concurrency level & bounded fan-out.
- Plan timeouts, cancellation & fallback strategy.
- Decide error handling & aggregation model.
- Preserve ordering requirements; apply backpressure.
- Propagate context (trace IDs, auth, locale).
- Ensure no blocking calls on event loop / threads.
- Observe with metrics, tracing, logs.
- Load test: validate limits, latency & failure modes.

10 SENIOR NOTES

- Async all the way: avoid sync-over-async deadlocks.
- Bound fan-out with semaphore / connection pool.
- Cancellation is a cooperative protocol, not preemption.
- Timeouts need cleanup and idempotent operations.
- Structured concurrency clarifies ownership & lifetime.
- Event-loop latency is your canary—find blocking.
- Prefer immutable data or proper synchronization.
- Retry with jitter; respect downstream rate limits.
- Separate CPU parallelism to dedicated executors.
- Measure queue depth & event-loop lag in production.
- Design APIs to surface cancellation & timeouts.

EVENT LOOP vs THREAD POOL (AT A GLANCE)



11 MENTAL HOOK

“ Async programming frees the execution thread while preserving the unfinished computation. ”